CS F214
Logic in CS
BITS Pilani, Hyderabad Campus
Assignment -1
Due Date : 15th October 2025 (by Midnight)
Total Marks: 30 (weightage : 10%)
Part-A

Objective:

The objective is to implement storing a propositional logic formula using a parse tree and run various algorithms you have learnt in this course on this representation of a propositional logic formula.

As first step you have to implement the construction of a parse tree for a propositional logic formula. You can assume that the formula is well formed and fully parenthesized. The input to your program should be a propositional logical formula in infix notation. The connective symbols you will use for this assignment is as follows.

1. ~ for negation
2. + for OR
3. * for AND
4. > for implication.

Your code should have **separate functions** to perform the given tasks.

Task 1:
Write a function to **convert the infix** propositional logic expression **into a prefix** propositional logic expression. **[2]**

Task 2:
Write a function to **convert the prefix expression into a rooted binary parse tree**. **[3]**

Task 3:
Write a function to traverse the parse tree to **output the infix expression** back by in-order traversal of the parse tree both recursively and iteratively. Implement pre-order and post-order traversal both recursively and iteratively. **[3]**

Task 4:
Write a function to **compute the height of a parse tree** both recursively and iteratively. **[2]**

Task 5:
Write a function to **evaluate the truth value of a propositional logic formula**, given the truth values of each propositional atom by traversing the tree in a bottom up fashion. Use it to produce the complete truth table for the propositional logic formula. **[3]**

Task 6:
Analyze your algorithms in terms of output, efficiency, and memory usage. Make sure that your code is well indented and commented. Document your code using Doxygen. Document your results and analysis as HTML pages (pay attention to the aesthetics).

You can use the below dataset(s) for all the tasks. **[5]**

Data Set: Refer to the SAT competition website:
https://satcompetition.github.io/

Pickup SAT 2002 benchmark. Download the compressed folder. (If you have download issues, please contact me).

You will find the files are in .cnf format. These are text files and hence can be opened using text editors.

Here is a website explaining the .cnf file format

https://jix.github.io/varisat/manual/0.2.0/formats/dimacs.html

References to look into:
http://scanftree.com/Data_Structure/infix-to-prefix

Part-B

Objective:
LEAN is an interactive theorem prover and functional programming language used to write formal, machine-checked mathematical proofs. Today, it has become an important verification tool for AI models.

The objective of this part of the assignment is to learn LEAN to write interactively proof of the sequent in propositional logic.

An Introduction to LƎ∀N

(prepared by Tobit K Joshy and Kanika Chandak)

> "Lean is an open-source programming language and proof assistant that enables correct, maintainable, and formally verified code"

Brief

Lean 4 is a formal proof assistant and programming language that can be used to express mathematical statements and construct proofs that are checked automatically by a computer. Unlike conventional mathematical proofs, where the validity of each step is checked by the reader, Lean verifies that every step

of a formal proof follows according to its underlying logical rules. This makes Lean useful for theorem proving and formal verification.

Lean is particularly useful for proving sequents in propositional logic. A sequent such as **P, Q ⊢ P ∧ Q** can be represented in Lean by treating **P** and **Q** as assumptions and **P ∧ Q** as the conclusion. The proof can then be constructed using tactics that correspond closely to the reasoning steps used in natural deduction.

```
theorem and_intro (P Q : Prop) : P → Q → P ∧ Q := by
intro hP
intro hQ
exact And.intro hP hQ
```

Here, **intro** introduces the assumptions **P** and **Q**, while **And.intro** applies the conjunction-introduction rule to construct a proof of **P ∧ Q**. Thus, Lean allows the logical rules studied in the course to be written explicitly and then mechanically verified.

Installation

Installing lean (using Visual Studio Code) for the purpose of this assignment is a simple process:

- **Step 1**

  Open VS Code.
  Go to the Extensions panel (Ctrl+Shift+X or Cmd+Shift+X).
  Search for **Lean 4** (by leanprover) and click **Install**.

- **Step 2**

  In VS Code, press Ctrl+N (or Cmd+N) to create a new file.
  Save it anywhere as <filename>.lean.
  A popup will appear in the bottom-right corner: *"Lean is not installed on this system"*.
  Click **"Install Lean using Elan"**

- **Final Step**

  Once the download finishes, the Lean Infoview side panel will appear automatically (or press Ctrl+Shift+Enter / click the ∀ icon in the top-right corner).

  Install — Lean Lang

Resources

LEAN language reference [Documentation]
How LEAN works [Youtube]

Tasks

Task 1: Your task is to prove Example 1.13-1.18 of the textbook using LEAN. **[5]**
Task 2: During Demo you will be asked to prove a sequent. **[7]**

General Instructions:

1. This assignment will be done in groups of max five students.
2. **Code must be written in C or C++ only for PART-A.**
3. There will be only one submission per group on the LMS.
4. The name of the file should be **id1_LOGIC_A1.zip**, where id1 refers to the BITS ID of the sender.
5. **You can discuss with your friends but refrain from copying the code and submitting.**
6. You have to demo the code to the instructor/TAs on a scheduled date and timing after submission. **It is important to attend the demo, as absence from demo will amount to zero credit for the assignment. The tentative date for the Demo will be right after the submission of the assignment.**